

UNIVERSIDAD AUTÓNOMA DE SINALOA

FACULTAD DE INGENIERÍA MOCHIS LICENCIATURA EN INGENIERÍA DE SOFTWARE



Administración de Sistemas

Tarea 4: SSH

Maestro. Herman Geovany Ayala Zuñiga

Alumno. Sañudo Zamorano Andrés Antonio

Fecha. 23 de febrero de 2026

1. CONTROL DE VERSIONES

Historial de Cambios

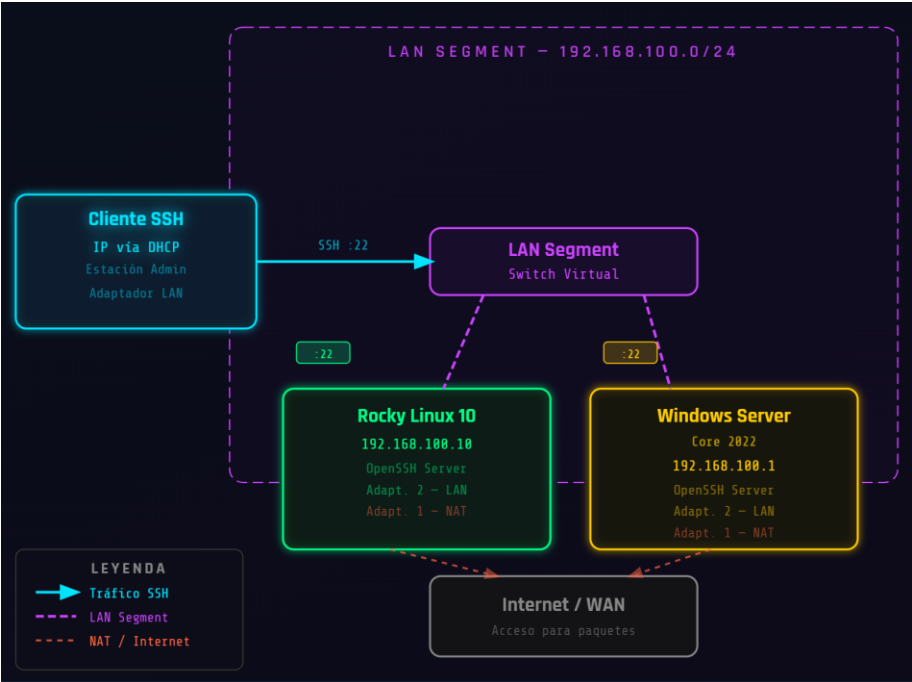
Versión	Fecha	Autor	Descripción de la Modificación
1.0	22/Feb/2026	andy934	Script instalación SSH
1.1	22/Feb/2026	andy934	Corrección de verificación del servicio SSH
2.0	22/Feb/2026	andy934	Script instalación SSH Windows
2.1	22/Feb/2026	andy934	Corrección de reglas del firewall en Linux y Windows

2. INTRODUCCIÓN Y ARQUITECTURA

2.1 Objetivo de la Práctica

El estudiante implementará y asegurará el servicio de acceso remoto SSH en ambos servidores (Linux y Windows)

2.2 Diagrama de Topología



3. GUÍA DE USO DE LOS SCRIPTS

3.1 Requisitos Previos

Para Linux (Rocky Linux 10):

- Usuario con privilegios sudo o acceso root
- Conexión a Internet o repositorios locales configurados
- Bash versión 4.0 o superior
- Espacio en disco: mínimo 50 MB

Para Windows (Server Core 2022):

- Usuario con privilegios de administrador
- PowerShell 5.1 o superior
- Política de ejecución: RemoteSigned o Unrestricted
- Conexión a Internet para descarga de componentes

3.2 Instrucciones de Ejecución

Linux:

Comando de ejecución:

```
sudo bash ssh-config.sh
```

Windows:

Comando de ejecución:

```
.\ssh-config.ps1
```

3.3 Flujo de Interacción

El script presenta un menú interactivo con las siguientes opciones:

Opción 1 - Instalar SSH:

No solicita datos adicionales. Ejecuta automáticamente:

- Verificación de instalación previa (idempotencia)
- Instalación de paquetes OpenSSH
- Habilitación del servicio en el boot
- Configuración automática del firewall

Opción 2 - Status:

Muestra el estado actual del servicio SSH (activo/inactivo, PID, memoria utilizada).

Opción 3 - Salir:

Finaliza la ejecución del script.

4. BITÁCORA DE DESARROLLO Y CONFIGURACIÓN

4.1 Explicación del Script - Linux

Lógica de Idempotencia

La función principal verifica si OpenSSH ya está instalado mediante consulta a la base de datos RPM. Si el paquete existe, el script termina con un mensaje informativo, evitando reinstalaciones que podrían sobrescribir configuraciones personalizadas.

Comando clave: `rpm -q openssh-server`

Propósito: Consultar si el paquete está en el sistema sin producir salida visual (redirección a `/dev/null`).

Proceso de Instalación

El script utiliza DNF (gestor de paquetes de RHEL/Rocky) para instalar dos componentes:

1. **openssh-server:** Demonio que escucha conexiones SSH entrantes en el puerto 22.
2. **openssh-clients:** Herramientas cliente (`ssh`, `scp`, `sftp`) para conexiones salientes.

La flag `-y` responde automáticamente 'yes' a todas las confirmaciones, permitiendo una instalación completamente desatendida.

Gestión del Servicio

Se utilizan dos comandos de `systemd`:

systemctl enable sshd: Crea un enlace simbólico en `/etc/systemd/system/multi-user.target.wants/` para que el servicio inicie automáticamente en cada arranque del sistema.

systemctl start sshd: Inicia el demonio inmediatamente sin esperar al siguiente reinicio del servidor.

Configuración del Firewall

Firewalld maneja las reglas de red mediante zonas. El script ejecuta dos operaciones:

1. **Agregar servicio SSH a la zona public:** La regla predefinida 'ssh' permite tráfico TCP entrante en el puerto 22.
2. **Recargar la configuración:** Aplica las reglas permanentes sin interrumpir conexiones existentes.

4.2 Explicación del Script - Windows

Verificación de Capacidad Opcional

Windows Server incluye OpenSSH como una 'capacidad opcional' que debe habilitarse explícitamente. El script consulta el estado de esta capacidad mediante el cmdlet Get-WindowsCapability, filtrando por nombre 'OpenSSH.Server*'.

Si el estado es 'Installed', el script termina para evitar reinstalaciones innecesarias.

Instalación mediante DISM

El cmdlet Add-WindowsCapability utiliza el framework DISM (Deployment Image Servicing and Management) para instalar OpenSSH.Server sin requerir reinicio del sistema.

Configuración del Servicio

Dos cmdlets configuran el comportamiento del servicio sshd:

Start-Service: Inicia el servicio inmediatamente.

Set-Service -StartupType 'Automatic': Configura el inicio automático en cada arranque del sistema operativo.

Regla de Firewall

El script verifica primero si ya existe una regla para OpenSSH. Si no existe, crea una regla de firewall con los siguientes parámetros:

- Nombre: OpenSSH-Server-In-TCP
- Dirección: Inbound (tráfico entrante)
- Protocolo: TCP
- Puerto local: 22
- Acción: Allow (permitir conexiones)

4.3 Evidencias de Configuración

Archivo generado (Linux): /usr/lib/systemd/system/sshd.service

Contiene la configuración del servicio SSH administrada por systemd.

Enlace simbólico creado: /etc/systemd/system/multi-user.target.wants/sshd.service → /usr/lib/systemd/system/sshd.service

Regla de firewall (Linux): Servicio 'ssh' agregado a la zona 'public' en /etc/firewalld/zones/public.xml

Regla de firewall (Windows): Visible en 'Windows Defender Firewall with Advanced Security' bajo reglas de entrada.

5. PRUEBAS DE FUNCIONAMIENTO

5.1 Protocolo de Pruebas - Linux

Prueba	Acción Realizada	Resultado Esperado	Resultado Obtenido
Instalación SSH	sudo bash ssh-config.sh, opción 1	Paquetes instalados sin errores	✓ OK
Servicio activo	systemctl status sshd	Estado: active (running)	✓ OK
Firewall configurado	firewall-cmd --list-services	Servicio 'ssh' en la lista	✓ OK
Conexión SSH remota	ssh usuario@192.168.100.10	Acceso al shell remoto	✓ OK

5.2 Protocolo de Pruebas - Windows

Prueba	Acción Realizada	Resultado Esperado	Resultado Obtenido
Instalación OpenSSH	.\ssh-config.ps1, opción 1	Capacidad instalada sin reinicio	✓ OK
Servicio corriendo	Get-Service sshd	Status: Running	✓ OK
Regla de firewall	Get-NetFirewallRule	Regla habilitada puerto 22	✓ OK
Conexión SSH remota	ssh usuario@192.168.100.20	Acceso a PowerShell remoto	✓ OK

5.3 Capturas de Validación

Windows a Cliente.

```
PS C:\Users\Administrador> ssh shadou@192.168.100.120
shadou@192.168.100.120's password:
Web console: https://Cliente-Linux-Sistemas.localdomain:9090/ or https://192.168.137.131:9090/

Last login: Mon Feb 23 16:04:55 2026
shadou@Cliente-Linux-Sistemas:~$
```

Cliente a Windows

```
Administrador: C:\Windows\system32\conhost.exe – ssh administrador@192.168.100.1

Microsoft Windows [Versión 10.0.20348.2113]
(c) Microsoft Corporation. Todos los derechos reservados.

administrador@SRV-WIN-SISTEMA C:\Users\Administrador>
```

Srv Linux a Cliente

```
shadou@Srv-Linux-Sistemas:~$ ssh shadou@192.168.100.120
shadou@192.168.100.120's password:
Web console: https://Cliente-Linux-Sistemas.localdomain:9090/ or https://192.168.137.131:9090/

Last login: Mon Feb 23 16:09:08 2026 from 192.168.100.10
shadou@Cliente-Linux-Sistemas:~$ _
```

Cliente a Srv Linux

```
shadou@Cliente-Linux-Sistemas:~$ ssh shadou@192.168.100.10
shadou@192.168.100.10's password:
Web console: https://Srv-Linux-Sistemas:9090/ or https://192.168.137.128:9090/

Last login: Mon Feb 23 16:08:28 2026
shadou@Srv-Linux-Sistemas:~$
```

6. CONCLUSIONES Y REFERENCIAS

6.1 Lecciones Aprendidas

Problema 1: Duplicación de paquete en comando de instalación

Descripción: El comando original instalaba openssh-server dos veces en la misma línea.

Impacto: DNF ignora automáticamente los duplicados, por lo que no hubo errores funcionales, pero el código era redundante.

Solución: Eliminar la duplicación dejando: `sudo dnf install openssh-server openssh-clients -y`

Problema 2: Firewall bloqueando conexiones SSH

Descripción: En las pruebas iniciales, las conexiones SSH desde el cliente eran rechazadas a pesar de que el servicio estaba activo.

Causa raíz: El firewall de Linux (firewalld) estaba activo pero no tenía reglas para permitir el servicio SSH.

Solución: Agregar el servicio SSH a la zona public y recargar el firewall. Esto se documentó en el commit 'Corrección de reglas del firewall en Linux y Windows'.

Problema 3: Verificación incorrecta del estado del servicio

Descripción: La versión inicial del script no validaba correctamente si el servicio SSH estaba realmente activo después de la instalación.

Solución: Se implementó una verificación robusta mediante `systemctl is-active` y códigos de salida. Documentado en el commit 'Corrección de verificación del servicio SSH'.

6.2 Referencias Bibliográficas

1. Microsoft Learn - OpenSSH for Windows

https://learn.microsoft.com/en-us/windows-server/administration/openssh/openssh_install_firstuse?tabs=powershell&pivots=windows-server-2022

2. Master Da Web - Firewalld en AlmaLinux y Rocky Linux

<https://masterdaweb.com/es/blog/firewalld-en-almalinux-y-rocky-linux>

3. Rocky Linux Documentation - SSH CA Key Signing

https://docs.rockylinux.org/guides/security/ssh_ca_key_signing/?h=ssh

4. Hostinger - ¿Qué es SSH?

<https://www.hostinger.com/es/tutoriales/que-es-ssh>

6.3 URL repo

<https://github.com/andy934/Administracion-de-Sistemas.git>